

Auditoria do Sistema de Bônus

12 Níveis & Motor de Distribuição — Quanta Shop

Data: 20/03/2026

Nota de Qualidade do Módulo

3 / 10

O sistema de lançamentos e bônus está em pior estado do que o módulo AWIN. Regras de negócio espalhadas em 3 camadas distintas (C#, stored procedures, SQL Agent Job), percentuais não configuráveis sem deploy, e parte dos bônus rodando completamente fora do controle da aplicação.

Tipos de Bônus Registrados no Sistema (16 tipos)

Chave	Descrição
CHBK	Cashback externo (AWIN)
DTCSH	Cashback residual para upline
BDOB	Bônus duplo cashback
BRCD	Bônus duplo residual
BDIN	Bônus compressão dinâmica
CHBLF	Cashback de loja física credenciada
DCPLJ	Cashback residual de loja física
DTBAD	Bônus adesão
DTAD	Bônus residual adesão
PPOC	Bônus credenciamento
DPPOC	Bônus residual credenciamento
LM	Lançamento manual
EST	Estorno
SQ	Saque
DTSLD	Taxa de compra de saldo
ASNT	Assinatura

Problemas Críticos Identificados

1. Regra de negócio dentro do Repositório

Os percentuais dos 12 níveis estão em um switch/case dentro de `LancamentoRepositorio.cs` — que é uma camada de acesso a dados, não de negócio. Para alterar um percentual de comissão, é necessário recompilar e fazer deploy da aplicação inteira.

Trecho do código atual:

```
case 1:
    valorPorNivel = 5m / 100m * valorTotalDistribuido; // 5% — hardcoded
case 2: ... case 10:
    valorPorNivel = 0.5m / 100m * valorTotalDistribuido; // 0.5% — hardcoded
case 11:
case 12:
    valorPorNivel = 0.25m / 100m * valorTotalDistribuido; // 0.25% — hardcoded
```

2. Mapeamento Produto != Número de Níveis também hardcoded

Em `LancamentoNegocio.cs`, a quantidade de níveis por produto está fixa no código (Produto 1: 0 níveis, Produto 2: 3 níveis, ..., Produto 6: 12 níveis). O banco já possui a tabela `ProdutoNivel` para guardar exatamente isso — mas o código C# não a utiliza.

3. N+1 queries para buscar corrente de upline

O método `ListaUsuariosPatrocinadores` faz uma consulta ao banco por nível em loop iterativo. Para um usuário com 12 níveis de upline, são 12 consultas sequenciais antes de qualquer distribuição começar — repetido para cada transação processada.

4. Busca de usuário do sistema dentro de transação financeira

Uma consulta ao banco pelo login literal "triumph" acontece dentro de cada transação de lançamento manual. Se esse usuário for renomeado ou removido, todo o processo de distribuição quebra silenciosamente.

5. Stored procedures órfãs — bônus binário fora do controle da aplicação

spc_processabinario e spc_DistribuicaoCashback não são chamadas por nenhum código C#. São executadas por SQL Agent Job diretamente no banco — fora do versionamento do código, fora dos logs da aplicação, fora de qualquer rastreabilidade do sistema. Parte dos bônus pagos não passa por nenhum controle da aplicação.

6. Legado Zanox ainda vivo

spc_DistribuicaoCashback ainda possui o parâmetro @nomeProgramaZanox. A Zanox foi renomeada para AWIN em 2017. Esse procedimento nunca foi modernizado.

Cashback externo (AWIN)

CashbackNegocio.LancarCashback !' spc_LancarCashback

LancamentoNegocio != LancamentoRepositorio (switch/case C#)

Compra de saldo

Plano adquirido

spc_DistribuicaoPlanoAdquirido (SQL Agent Job)

spc_processabinario (SQL Agent Job — sem chamada C#)

Plano de Ação Recomendado

1. Criar tabela ConfiguracaoNivel para guardar percentual por nível e por produto, eliminando os hardcodes
2. Mover lógica de distribuição do Repositório para a camada de Negócio
3. Substituir o N+1 por uma única query com CTE recursiva
4. Chamar spc_processabinario a partir do Worker C# — trazer o bônus binário para o ciclo rastreável
5. Remover referências a ZanoX nas procedures e modernizar parâmetros

Relatório gerado automaticamente a partir de análise de código fonte da
Quanta Shop.